

CSCI3060u Lab

THINGS TO ADD:

Additional from in class we need to add: Add a transaction LIST that outputs to terminal all items currently up for sale and relevant information (including on sale, and costs)

Each input / output file is just attributes, output would also have Status: FAILURE or SUCCESS, and Account Created: No or Yes

Separate functions into create (creating accounts), add credit (for users adding to credit for account), buy (for purchasing)

Test plan document:

We built a directory that is representative of the application's complete front end. Within each component of the directory, the corresponding tests of each component will be found. This ensures it is easy to see the tests affecting a code component, since they will be found in the same folder as the code files they validate. This also helps to identify issues with directory structure before implementation.

The directory structure is as follows: Tests > Feature Family* > Required Features > Fail/Success Route > Fail/Success Condition > Input/Output/Description (for example, Tests \ buying_process \ adding_to_cart \ failure_routes \ product_unavailable leads to a folder with the failed input, the output that reads as a failure, and a description of the failure condition for a product being unavailable when trying to add it to your cart). A Feature Family represents a major area of the application, such as buying_process, changing_credit, or create_account_info. Required Features represent specific functions under a family, such as inputting_payment under changing_credit. Each test is divided into failure and success routes that contain the various conditions for failure or success. Each condition has a specific input to test our product, a specific failure or success output for testing/debugging, and a description in plain english of what the test is meant to verify.

The test files will be Python files that are run with pytest, and should be run each time a code component is updated or added. These tests use specific input to verify that each test case is able to function properly and give a successful output. For example, given the input [REDACTED] for the test [REDACTED] a failure would return [REDACTED] and a success would return [REDACTED]. This process will be automated using shell scripts (.bat file, as this is developed in a windows environment) rather than having each test manually performed each time a change is made to the program.

Output of the tests will be stored in an "output" directory folder with specific error codes that help with logging and debugging. Each test run will be logged with a new name (Run_0001, Run_0002) in order to easily identify most recent runs and not run into file name conflicts, and to

allow previous runs to be retained to identify differences in output between program versions. The output files will be stored as plain text. The test directory will also contain a text file with all test inputs, descriptions, and expected outputs to allow for easier debugging and problem visibility.

Therefore, the test process will involve:

1. Creation/modification of feature/component
2. Identification of possible errors or problems
3. Test cases created for said expected errors
4. Implementation of test cases
5. Test cases added to pytest
6. Pytest ran to identify any errors
7. Repeat upon further modification

Test Cases:

- Root directory
- Input
- Output
- Description
 - assets
 - Input
 - Output
 - Description
- server
 - Input
 - Output
 - Description

Testing Categories